Indoor Positioning via Wi-Fi Signal Strength Features and Classical Machine Learning

Project Signal2Space

Abstract

This report evaluates Wi-Fi RSSI fingerprints for building, floor, and two-dimensional coordinate prediction using UJIIndoorLoc. The pipeline uses grouped development data, training-only signal preprocessing, and separately tuned Random Forest models. On the supplied 1,111-record held-out partition, building accuracy is 99.73%, floor accuracy is 88.93%, and mean Euclidean coordinate error is 11.58 units. These results apply to this partition; overlap may remain across splits, and the coordinate reference system is unverified.

Machine Learning Scope

Covers data validation and EDA, leakage-aware feature preparation, separate building/floor classifiers and coordinate regression, internal tuning, held-out evaluation, and integrated error analysis. Reusable features, predictions, metrics, confusion matrices, and figures are produced.

Project at a Glance

Dataset	19,937 training/reference records; 1,111 supplied held-out records.
Inputs	520 WAP RSSI fields, expanded to 1,046 engineered predictors.
Models	Three independent Random Forest estimators for building, floor, and two-coordinate prediction.
Validation	Grouped internal split for tuning; supplied validation reserved for final evaluation.

Keywords: indoor positioning, Wi-Fi fingerprinting, RSSI, Random Forest, classification, coordinate regression, UJIIndoorLoc

Contents

1 Introduction **3**
- 1.1 Objectives . . . 3

2 Data and problem definition **3**
- 2.1 Dataset and observed schema . . . 3
- 2.2 Signal encoding and quality checks . . . 4

3 Pipeline overview **5**

4 Data partitioning and leakage controls **6**
- 4.1 Development split . . . 6
- 4.2 Why an internal validation set is used . . . 6
- 4.3 Predictor allowlist and fitting boundaries . . . 7

5 Preprocessing and feature engineering **7**
- 5.1 Missing-aware RSSI transformation . . . 7
- 5.2 Signal summary features . . . 8

6 Modeling methods **8**
- 6.1 Random Forest rationale . . . 8
- 6.2 Building classification . . . 9
- 6.3 Floor classification . . . 9
- 6.4 Coordinate regression . . . 10

7 Evaluation measures **11**

8 Results **11**
- 8.1 Task-level performance . . . 11
- 8.2 Building and floor classification . . . 13
- 8.3 Coordinate regression . . . 14
- 8.4 Hierarchical evaluation . . . 16

9 Notebook-by-notebook implementation **17**
- 9.1 ML_01: data preparation, EDA, and feature export . . . 17
- 9.2 ML_02: building classification . . . 18
- 9.3 ML_03: floor classification . . . 18
- 9.4 ML_04: coordinate regression . . . 19
- 9.5 ML_05: integrated evaluation and consistency checks . . . 19
- 9.6 Artifact locations and reproducibility . . . 19

10 Discussion **20**

11 Limitations **20**

12 Conclusion **20**

1 Introduction

Indoor satellite positioning is often unavailable or unreliable, while Wi-Fi access points provide signal observations that vary across locations. A Wi-Fi fingerprint is a vector of received signal strengths measured from a set of access points. The project treats each row as a fingerprint and predicts three related location targets: building identity, floor identity, and two-dimensional position.

The implemented system uses a shared, deterministic feature preparation pipeline followed by three independent tabular estimators. This structure follows the project specification: absent access points are represented consistently, signal-density summaries are added, classification predicts building and floor, and regression predicts coordinates. The UJIIndoorLoc database was introduced for multi-building and multi-floor WLAN fingerprint-based localization research [1, 2].

The central modeling question is: how well can a Wi-Fi fingerprint predict each location target under the supplied data partition when the transformations and model settings are learned without using the final held-out labels? The three prediction targets are evaluated separately because building class, floor code, and coordinates have different label types and error measures. Their predictions are then joined to describe whether their errors coincide on the same records.

1.1 Objectives

The report addresses the following objectives:

- describe and validate the data used by the implemented pipeline;
- define the partitioning and preprocessing protocol and its leakage controls;
- document the engineered signal features and selected model configurations;
- report classification, regression, and hierarchical evaluation results; and
- state limitations that affect interpretation and reproducibility.

2 Data and problem definition

2.1 Dataset and observed schema

The repository contains the UJIIndoorLoc training/reference file and supplied validation file. The observed training file has 19,937 rows and 529 columns; the supplied validation file has 1,111 rows and 529 columns. Each file contains 520 WAP signal columns, four targets (building ID, floor, longitude, latitude), and five metadata fields (space ID, relative position, user ID, phone ID, timestamp). Dataset documentation describes measurements from multiple buildings and floors at Universitat Jaume I [2, 1].

Table 1 summarizes how these columns are used. A row is a measurement record, not a sequence model input. The WAP feature names are ordered by their numeric suffix; row identifiers and collection metadata are carried through for traceability and split checks, not supplied to the estimators.

Table 1: Observed data groups and their roles in the pipeline.

Column group	Count	Pipeline role
WAP RSSI fields	520	Inputs; sentinel-aware scaling, presence masks, plus row summaries.
Building and floor	2	Classification targets; never included in the predictor allowlist.
Longitude and latitude	2	Joint regression targets; never included in the predictor allowlist.
Collection/location metadata	5	Retained for auditing, grouping, and interpretation; excluded from predictors.

The targets are treated as three separate prediction tasks. Building and floor are integer-coded classes. Longitude and latitude are modeled jointly as two numeric outputs. Floor codes are reused across buildings, so floor classification predicts the floor code alone; it is not a single combined building–floor class. The held-out validation file contains zero placeholders for `SPACEID`, `RELATIVEPOSITION`, and `USERID`; these metadata fields are not used as model inputs.

2.2 Signal encoding and quality checks

The notebooks discover WAP columns by matching the `WAP` prefix and numeric suffix. In both source files, the value `100` is the dominant missing-signal sentinel; other observed signal values are non-positive and are interpreted as dBm. Training data has an average absent-signal fraction of about 96.54% across WAP entries, compared with about 96.83% in the supplied held-out data. There are no blank cells in the checked files. The training file contains 637 exact duplicate rows and 76 rows with no detected access points; these rows were counted and retained. The held-out file has no exact duplicate rows or all-absent-signal rows in the recorded checks. A full-row hash comparison found no exact raw-row overlap between the two supplied files.

The class counts are not uniform. There are three building classes and five floor codes in the training data, while floor 4 has substantially fewer examples than floors 0–3. The validation partition also has only 39 floor-4 examples. This imbalance is relevant when interpreting raw accuracy: a model can achieve high overall accuracy while still making a disproportionate number of errors on a small class.

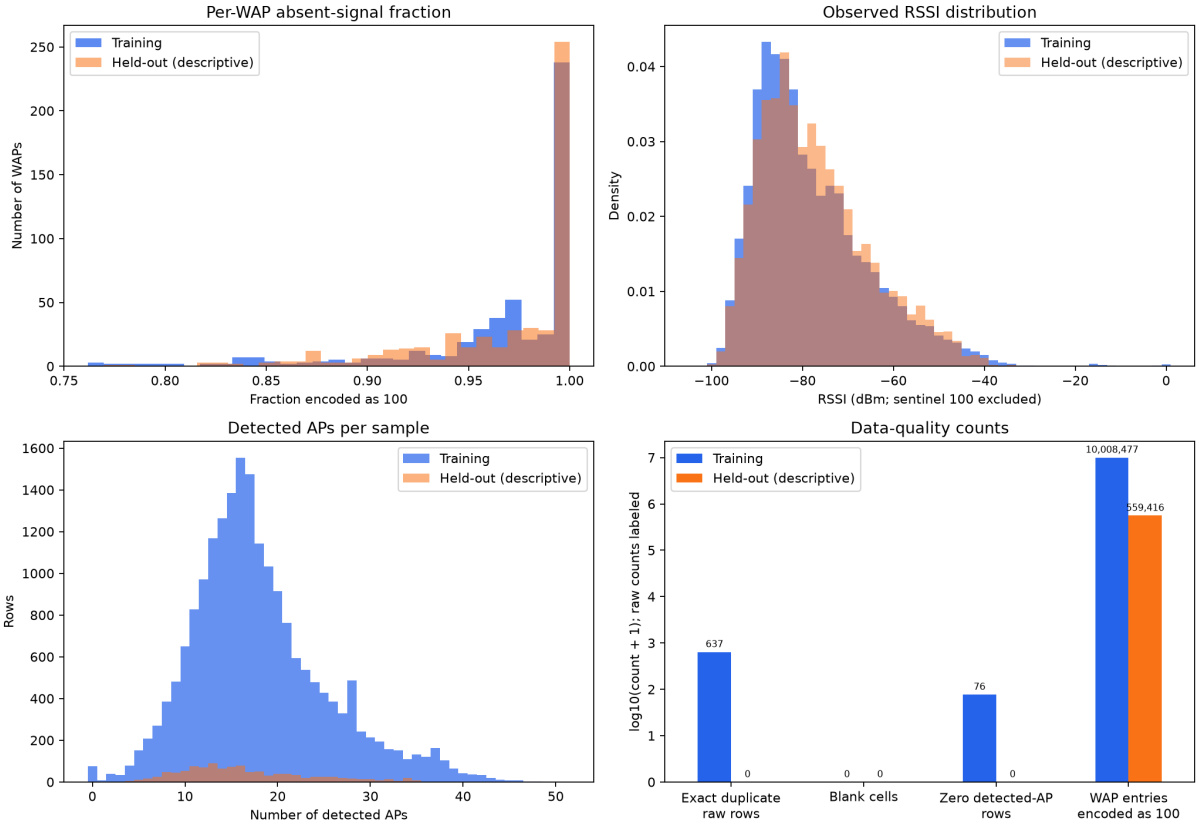


Figure 1: Exploratory data quality and signal distributions. The sentinel value is excluded from the observed RSSI histogram. Held-out distributions are shown descriptively; they are not used to fit preprocessing or select models.

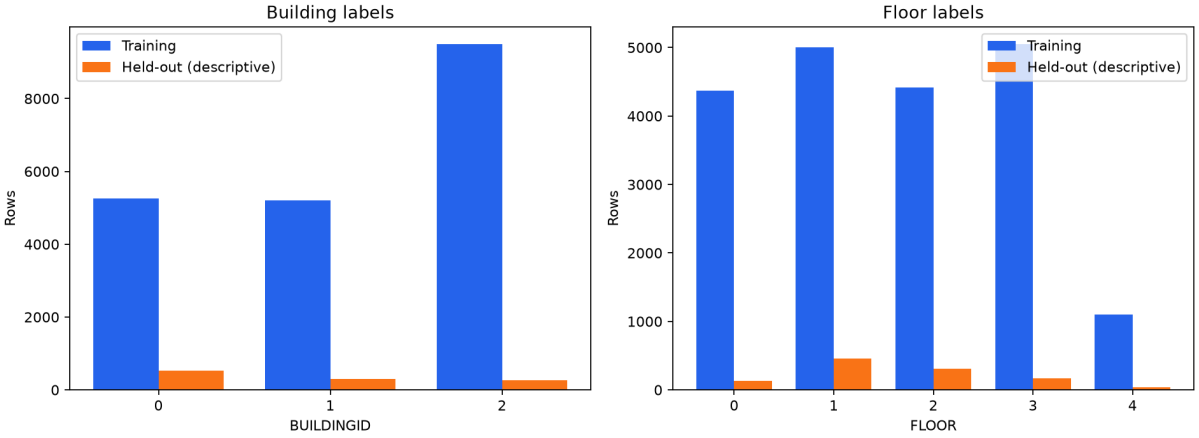


Figure 2: Building and floor label counts in the training and supplied held-out files. Held-out bars are descriptive only; they are not used for model selection. The small floor-4 support motivates reporting macro F1 and class-specific confusion counts alongside accuracy.

3 Pipeline overview

The implemented workflow comprises five notebooks in numerical order. The first produces the shared feature tables; the next three train the task estimators; the fifth integrates their row-level outputs and verifies that they refer to the same samples.

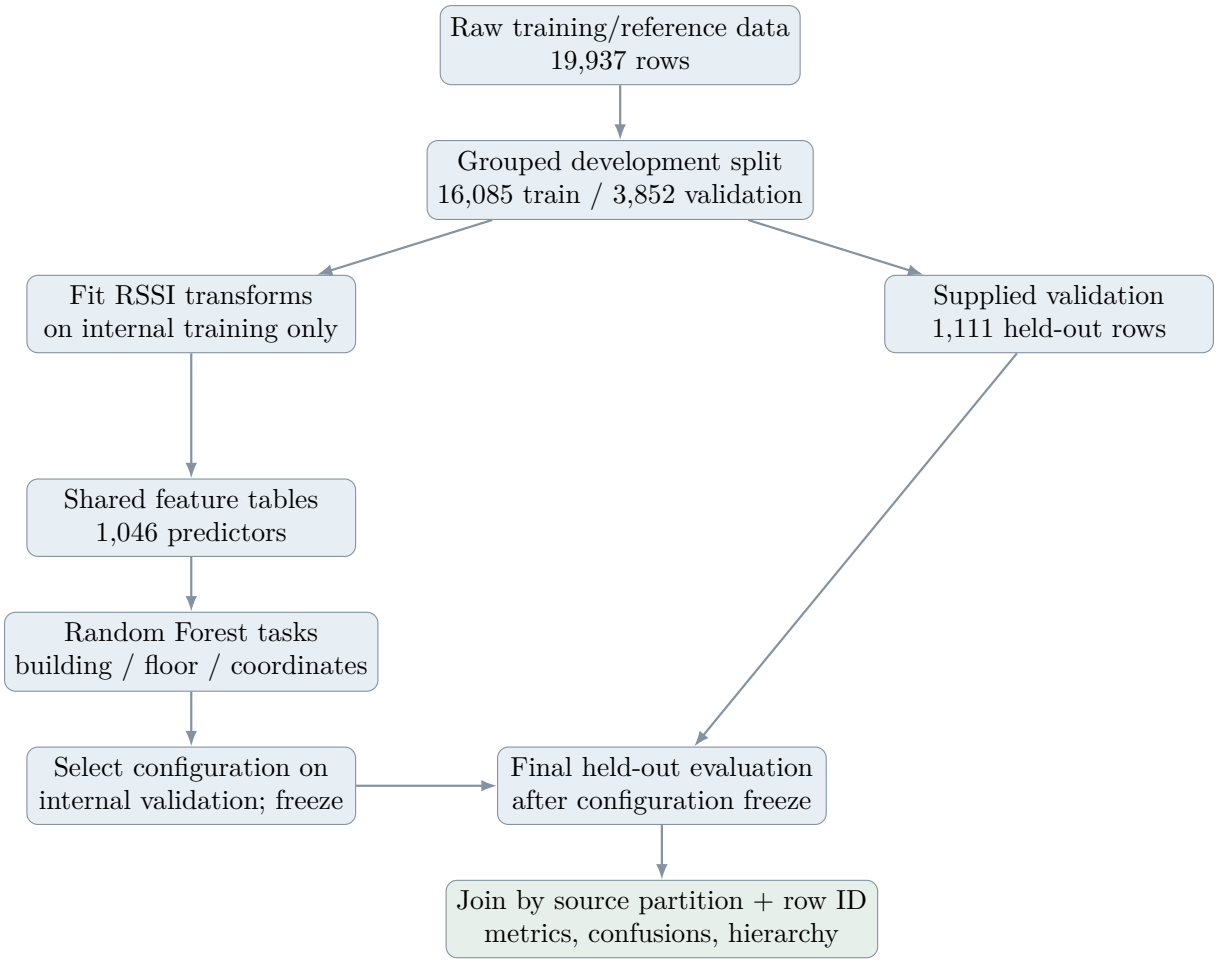


Figure 3: Implemented AML data and model flow. The provided validation file is held aside while transformations and model configurations are learned.

4 Data partitioning and leakage controls

4.1 Development split

The 19,937-row training/reference file is divided with `GroupShuffleSplit` using a 20% internal validation fraction and random seed 42. Groups are defined by the composite building, floor, and space identifiers. This gives 16,085 internal-training rows and 3,852 internal-validation rows; the two subsets have disjoint composite group sets. The original supplied validation file remains a separate 1,111-row held-out partition.

The split is made on a composite key, not on individual rows. The notebook verifies 588 unique groups in internal training and 147 in internal validation, with no group shared between them. This prevents the same building/floor/space key from being split across these two development subsets. It also verifies unique row IDs across the internal split artifacts.

4.2 Why an internal validation set is used

The supplied `validationData.csv` is reserved as the final evaluation partition. A second, internal validation subset is created from `trainingData.csv` because model selection still requires a labelled set on which candidate settings can be compared. If tuning choices were made by repeatedly checking the supplied held-out scores, information from the final evaluation labels

would influence the chosen model and the reported held-out score would become optimistically biased. The internal validation set handles model selection; the supplied validation set estimates the performance of the frozen configuration under the project's chosen split.

The internal validation rows are not used to fit the scaler or estimator. They are used to score candidate configurations and choose among them. The selected estimator is then fit on the internal-training rows and applied once to the supplied held-out rows. This evaluation estimates performance for the supplied partition; it is not an independent external-site test, and it does not guarantee that signal fingerprints or 2D coordinates are unique across all partitions.

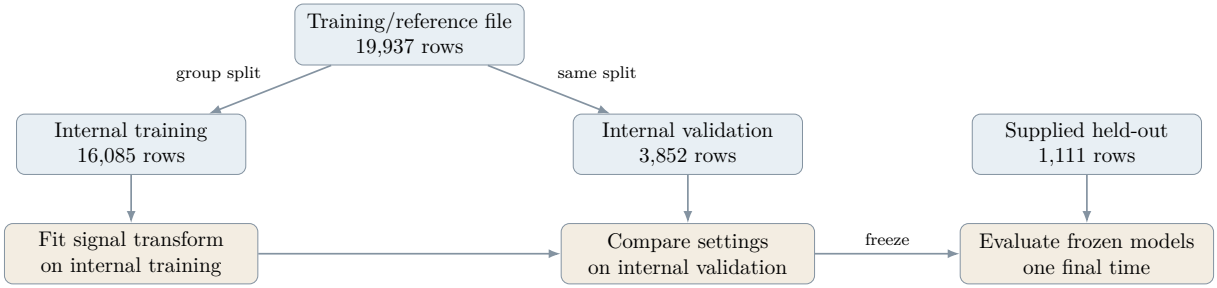


Figure 4: Development and final evaluation roles. The internal validation rows inform configuration selection; the supplied held-out rows do not inform feature fitting or tuning.

4.3 Predictor allowlist and fitting boundaries

The shared predictor matrix contains 1,046 columns: 520 scaled RSSI values, 520 binary detection indicators, and six signal summaries. Target labels and all identifiers/collection metadata are retained for verification and analysis but excluded from prediction. Each task notebook independently reconstructs and validates the feature allowlist from the feature table schema. RSSI scaling parameters are learned only from observed values in internal training and reused unchanged on internal validation and held-out data. The held-out file is loaded only after the task-specific configuration is frozen.

These checks prevent direct target or identifier leakage and prevent transformation statistics or hyperparameter selection from using held-out targets. They do not establish a stricter unseen-fingerprint or unseen-position evaluation. Comparing the exported WAP signal vectors shows eight distinct exact fingerprints shared between internal training and internal validation, and none shared between internal training and the supplied held-out partition. Exact coordinate pairs recur across both comparisons: 62 between internal training and internal validation, and eight between internal training and supplied held-out data. Thus, a complete row or fingerprint can be absent from a split while the same coordinate is represented in training. A grouped split by building/floor/space also does not assert independence of every underlying signal pattern.

5 Preprocessing and feature engineering

5.1 Missing-aware RSSI transformation

For each WAP, the sentinel value 100 is converted to an absent observation, not treated as a signal strength. For WAP j, the internal-training observed values define the mean μ_j and population standard deviation σ_j. Observed values are standardized as

$$z_{ij} = \frac{r_{ij} - \mu_j}{\sigma_j},$$

where r_{ij} is the RSSI of sample i for WAP j. A zero or undefined scale is replaced with one. Absent values receive a finite per-WAP fill below the minimum standardized training observation; a separate binary indicator remains authoritative for whether that WAP was actually seen. For WAPs with no observed internal-training samples, the transformation uses the observed internal-training RSSI pool as a fallback. No statistic is estimated from internal validation or held-out data.

5.2 Signal summary features

Six row-level features supplement the 1,040 WAP-specific inputs:

- `n_heard`: number of detected access points;
- `max_rssi_dbm`: strongest observed signal;
- `mean_rssi_dbm` and `std_rssi_dbm`: mean and standard deviation among detected signals;
- `n_strong`: number of observed signals stronger than -70 dBm; and
- `top1_ap_index`: zero-based index of the strongest WAP.

Rows with no detected WAP use explicit finite summary values and a strongest-access-point index of -1. The canonical output tables also retain row ID, source partition, original metadata, and all targets for traceability and evaluation joins. These retained fields are not part of the predictor allowlist.

The resulting feature row has 1,046 values. The WAP signal columns preserve relative per-access-point signal variation after normalization; the binary masks separately represent observation availability; the six summaries provide compact indicators of how many signals were detected and their overall strength. The explicit feature allowlist makes this predictor boundary inspectable and prevents a row key, source label, target, or metadata field from entering a model by accident.

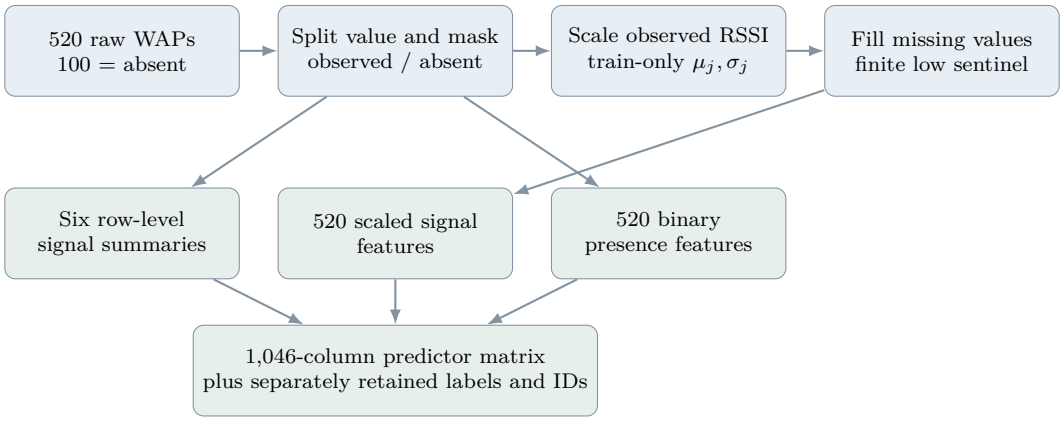


Figure 5: Feature construction from sparse WAP measurements. The mask preserves missingness explicitly, while all learned scaling values come from internal training only.

6 Modeling methods

6.1 Random Forest rationale

The project uses Random Forests for the three AML tasks. A Random Forest aggregates decision trees fit on bootstrap samples and randomized feature subsets [3]. At each split, a tree considers a subset of candidate predictors rather than always comparing all 1,046 inputs. This decorrelates

the trees; bootstrapping creates variation in their training examples. For classification the forest returns the most-voted class, while for regression it averages tree predictions for each output coordinate:

$$\hat{c}(\mathbf{x}) = \text{mode}_{b=1,\dots,B}\, h_b(\mathbf{x}), \qquad \hat{\mathbf{y}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \mathbf{g}_b(\mathbf{x}).$$

Here, B is the number of trees, h_b is a class prediction, and $\mathbf{g}_b$ is a two-coordinate prediction. Tree count, depth, leaf size, and feature subsampling shape the bias/variance trade-off: more trees stabilize the aggregation, while depth and leaf size control how finely individual trees partition the feature space. Class weights alter the contribution of floor classes during tree fitting. The project benchmarks Random Forest configurations only; it does not claim superiority over untested alternatives.

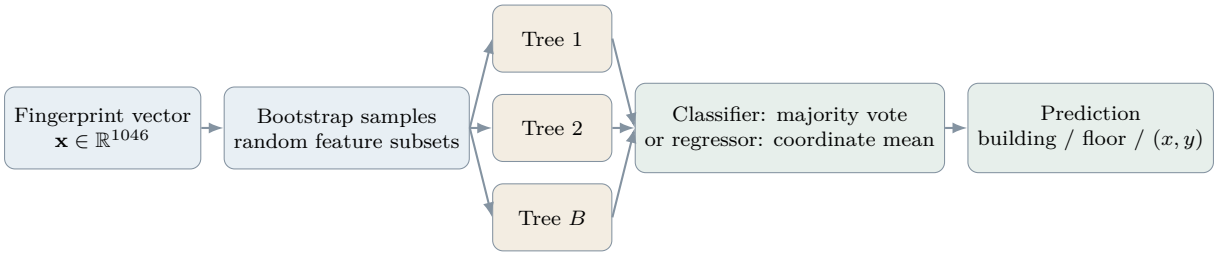


Figure 6: Prediction logic shared by the three Random Forest tasks. The target type determines how the individual tree outputs are aggregated.

All notebooks fix the random seed for reproducibility and run trees with parallel jobs. The selected parameter settings below are specific to this dataset and split. They are not universal defaults or evidence that these models are optimal beyond the tested grids.

6.2 Building classification

The building estimator is a `RandomForestClassifier` with `BUILDINGID` as its target. The notebook fits a 100-tree baseline first, then evaluates an 18-configuration grid on the internal validation rows. The grid varies tree count over $\{100, 200, 300\}$, maximum depth over $\{\text{unlimited}, 20, 30\}$, and minimum leaf size over $\{1, 2\}$. Each candidate is fit only on internal training. Internal-validation accuracy is the primary selection score, with macro F1 breaking ties. The selected setting is 100 trees, unlimited depth, and minimum leaf size one; it matches the recorded baseline setting. After selection is frozen, the estimator is fit on internal training and scored on the held-out file. Evaluation includes accuracy, macro F1, and a confusion matrix over the building labels observed in internal training.

6.3 Floor classification

The floor estimator is a separate `RandomForestClassifier` with `FLOOR` as its target. The observed floor-label mapping is formed from internal-training labels in ascending numeric order and reused for internal validation and held-out evaluation. Building ID and all metadata are excluded from predictors; therefore, this task predicts the floor code alone, not a building–floor pair. The model uses balanced class weights to reduce the influence of the uneven class frequencies during tree construction.

The eight-candidate grid crosses tree counts $\{150, 300\}$, depths $\{\text{unlimited}, 30\}$, and leaf sizes $\{1, 2\}$, with `max_features=sqrt` fixed. A 100-tree baseline is also reported. Internal-validation accuracy selects the configuration; macro F1 is a class-balanced diagnostic and is included in

deterministic tie-breaking. The chosen setting has 150 trees, unlimited depth, minimum leaf size one, `max_features=sqrt`, and balanced class weights. The held-out confusion matrix uses the fixed internal-training class list, so classes do not disappear when a partition has few examples.

6.4 Coordinate regression

The coordinate estimator is a multi-output `RandomForestRegressor` that predicts longitude and latitude together. Its explicit predictor allowlist excludes both coordinate targets and all metadata identifiers. The baseline uses 100 trees, unlimited depth, and leaf size one. Ten candidates then vary tree count (100, 150, or 200), maximum depth (unlimited or 50 for selected candidates), minimum leaf size (one or two), and feature subsampling (all features, `sqrt`, or 0.5). Each candidate is fit on internal training and scored on internal validation by mean Euclidean distance. The selected configuration uses 150 trees, unlimited depth, minimum leaf size one, and `max_features=0.5`. It reduces the internal-validation mean Euclidean error from 9.26 for the baseline to 8.58 coordinate units.

Table 2 reports every tested coordinate configuration, ranked by the internal-validation selection metric. The first row is the selected configuration. This table makes clear that selection was performed over the recorded ten candidates rather than over an unreported search.

Table 2: Coordinate regressor search results on internal validation, ranked by mean Euclidean error.

Trees	Maximum depth	Minimum leaf	Features per split	Mean Euclidean error
150	unlimited	1	0.5	8.5786
200	unlimited	1	sqrt	8.6567
150	unlimited	1	sqrt	8.6749
150	50	1	all	9.1912
150	unlimited	1	all	9.2004
200	unlimited	1	all	9.2024
200	50	1	all	9.2030
100	unlimited	1	all	9.2561
150	unlimited	2	all	9.4175
200	unlimited	2	all	9.4294

The coordinate notebook labels coordinates as EPSG:3857 projected metres. The repository does not provide authoritative CRS metadata, so all coordinate-unit results below must be read as values under the notebook's stated EPSG:3857 assumption. They are not corrected for projection scale and should not be interpreted as verified ground-distance errors.

Table 3: Task-specific search spaces and selection criteria. All candidates use the shared internal-training and internal-validation partitions.

Task	Parameters varied	Candidates	Selection rule
Building	Trees: 100, 200, 300; depth: unlimited, 20, 30; leaf: 1, 2	18	Accuracy; macro F1 tie-break
Floor	Trees: 150, 300; depth: unlimited, 30; leaf: 1, 2; features fixed to sqrt; balanced class weights	8	Accuracy; macro F1 and deterministic parameter-order tie-break
Coordinates	Trees: 100, 150, 200; depth: unlimited or 50; leaf: 1 or 2; features: all, sqrt, or 0.5 in the tested combinations	10	Lowest mean Euclidean error

7 Evaluation measures

For classification, accuracy is the proportion of correct predictions,

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}(y_i = \hat{y}_i),$$

while macro F1 is the arithmetic mean of per-class F1 scores,

$$\text{Macro F1} = \frac{1}{K}\sum_{k=1}^{K} \frac{2\,\text{Precision}_k\,\text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}.$$

Each of the K classes contributes equally to macro F1, so a sparse class is not overwhelmed by larger classes. The confusion matrix reports true labels by predicted labels. The floor confusion matrix is useful for examining neighboring-code confusions and the smaller number of floor-4 examples.

For coordinate predictions $(\hat{x}_i, \hat{y}_i)$ against observed coordinates (x_i, y_i), axis-specific mean absolute error is

$$\text{MAE}_x = \frac{1}{n}\sum_{i=1}^{n} |x_i - \hat{x}_i|, \qquad \text{MAE}_y = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|.$$

The two-dimensional error is $d_i = \sqrt{(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2}$; the report gives its mean and median. These are computed in the coordinate units recorded by the regression pipeline.

8 Results

8.1 Task-level performance

Table 4 reports internal-validation results used during development and final supplied held-out results. The held-out column is the main performance estimate; the internal-validation column reflects the model-selection data and is expected to be more favorable because candidates were ranked there. Table 5 makes the baseline-to-tuned change explicit. The supplied held-out evaluation is performed only after the selected configuration is frozen.

Table 4: Task metrics by split. Coordinate metrics use the notebook's stated EPSG:3857 metre assumption.

Task	Internal validation	Held-out validation	Selected model configuration
Building accuracy	99.95%	99.73%	RF: 100 trees, unlimited depth, leaf size 1
Building macro F1	99.95%	99.71%	same configuration
Floor accuracy	96.42%	88.93%	RF: 150 trees, unlimited depth, leaf size 1, `sqrt`, balanced weights
Floor macro F1	96.78%	87.27%	same configuration
Longitude MAE	5.91	8.10	RF regressor: 150 trees, unlimited depth, leaf size 1, max features 0.5
Latitude MAE	5.06	6.81	same configuration
Mean Euclidean error	8.58	11.58	same configuration
Median Euclidean error	6.31	7.49	same configuration

Table 5: Baseline, selected internal-validation, and final held-out metrics.

Task and metric	Baseline internal validation	Selected internal validation	Selected held-out evaluation
Building accuracy	99.95%	99.95%	99.73%
Building macro F1	99.95%	99.95%	99.71%
Floor accuracy	96.21%	96.42%	88.93%
Floor macro F1	96.58%	96.78%	87.27%
Coordinate mean Euclidean error	9.26	8.58	11.58

For building, tuning retains the baseline setting and scores are almost unchanged on the supplied partition. Floor tuning yields a modest improvement on internal validation; the difference between internal and held-out performance is larger, especially for floor classification. For regression, the selected configuration lowers mean Euclidean error by 0.68 coordinate units on internal validation compared with the baseline, while held-out error is higher than internal validation. The comparison is descriptive: the held-out data was not used to choose between configurations.

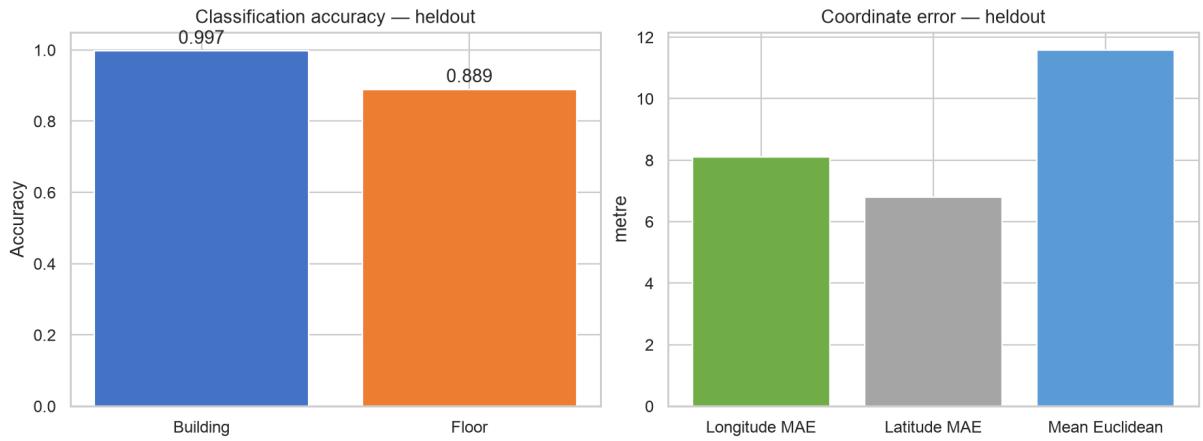


Figure 7: Held-out classification accuracy and coordinate error on their task-specific scales. The left and right panels have different metrics and units; bar heights should not be compared across panels.

8.2 Building and floor classification

Building classification correctly predicts 1,108 of 1,111 held-out rows. The building confusion matrix contains only three errors: one building-0 record and one building-1 record are predicted as a neighboring code, and one building-1 record is predicted as building 2. The floor classifier correctly predicts 988 of 1,111 rows. Most floor errors are between neighboring codes, and floor 4 has the smallest support. Table 6 reports per-code support, correct predictions, and recall derived from the held-out confusion matrix.

Table 6: Held-out floor recall by true floor code.

True floor	Support	Correct	Recall
0	132	119	90.15%
1	462	401	86.80%
2	306	279	91.18%
3	172	162	94.19%
4	39	27	69.23%

The largest single off-diagonal floor count is true floor 1 predicted as floor 2 (49 records), followed by true floor 4 predicted as floor 3 (10 records). Since floor codes are modeled as nominal classes, the classifier does not explicitly penalize a two-floor error more than a one-floor error; the matrix is needed to understand the structure behind accuracy and macro F1. Figure 8 presents the held-out building and floor matrices, and Figure 9 shows the floor counts at higher size.

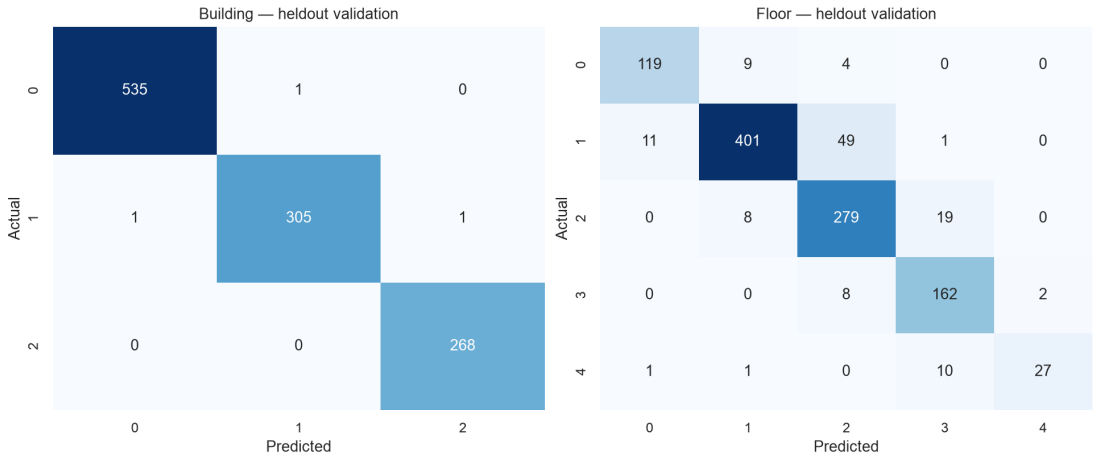


Figure 8: Held-out building and floor confusion matrices regenerated from the integrated row-aligned predictions.

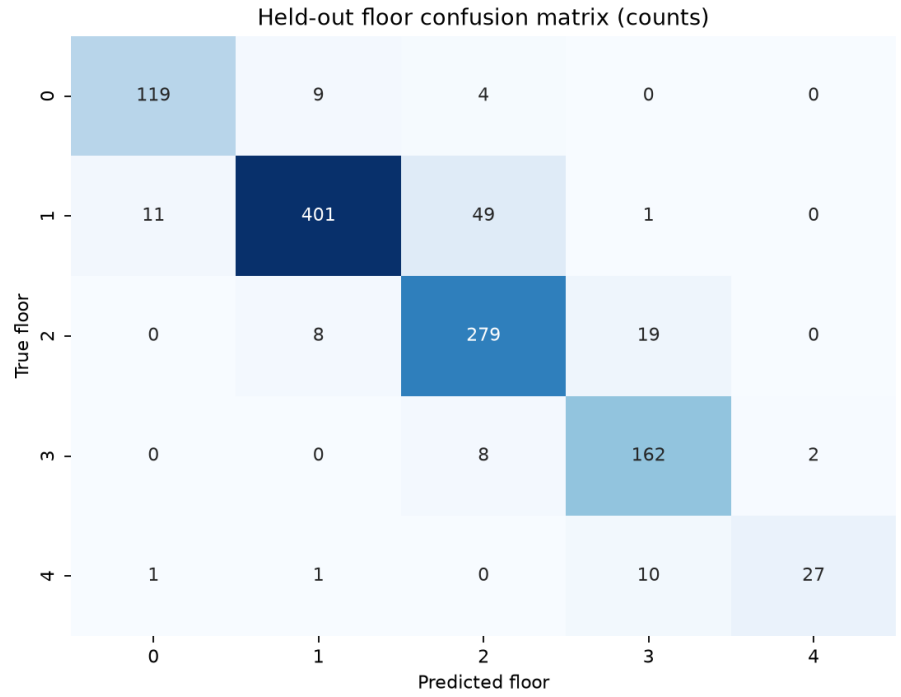


Figure 9: Held-out floor confusion matrix. Rows are true floor codes and columns are predicted floor codes.

8.3 Coordinate regression

The held-out longitude-axis MAE is 8.10 coordinate units, latitude-axis MAE is 6.81, mean Euclidean error is 11.58, and median Euclidean error is 7.49. The internal validation mean Euclidean error is 8.58. The increase on the supplied held-out partition indicates that internal-validation performance should not be substituted for the final estimate. Figure 10 compares actual and predicted coordinate values; Figure 11 shows held-out samples in coordinate space colored by prediction error. The visualization uses the recorded coordinate values directly and does not add a basemap or reproject the points.

The held-out error distribution is right-skewed: the median is below the mean, and a smaller set

of larger errors raises the mean. The 75th and 95th percentiles are 13.68 and 36.21 coordinate units, respectively. This distinction matters because the mean alone hides the error tail. Axis MAEs summarize residuals separately by coordinate component, while Euclidean error measures the combined two-dimensional displacement for each record.

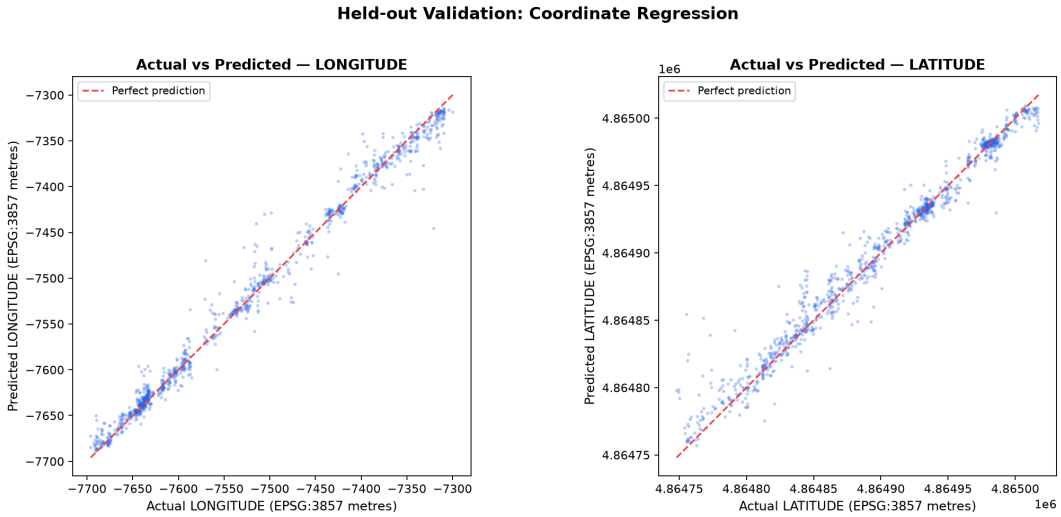


Figure 10: Held-out actual-versus-predicted coordinate plots produced by the regression notebook.

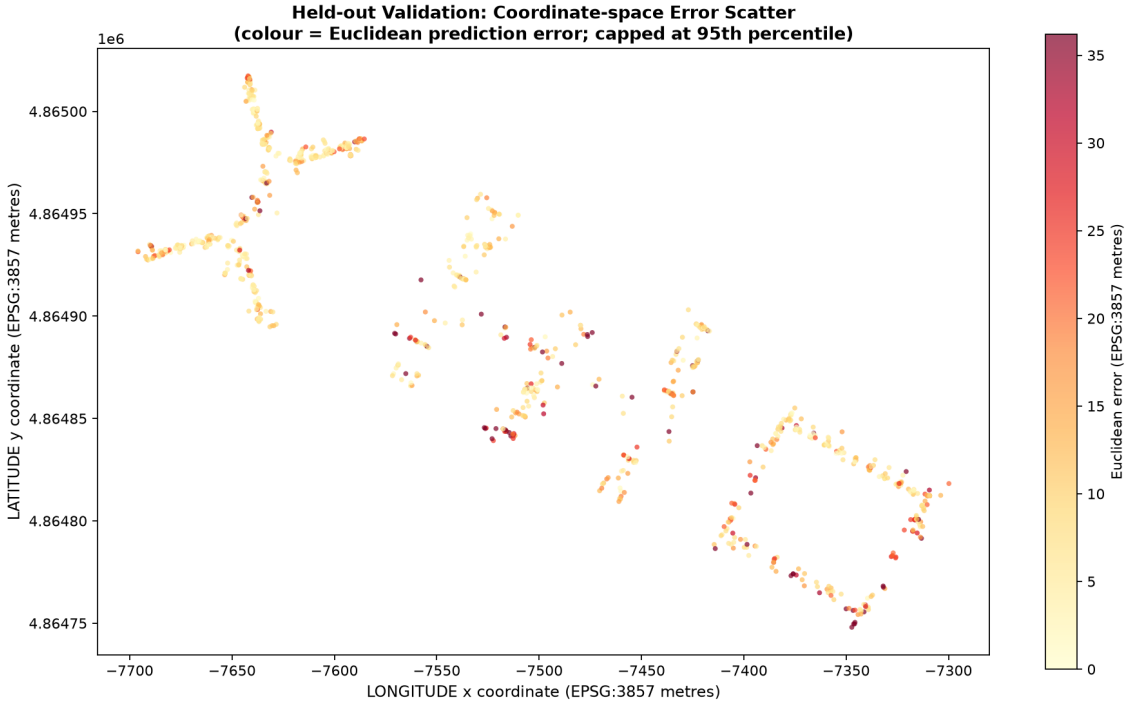


Figure 11: Held-out coordinate-space error scatter. Color encodes Euclidean prediction error; axes and error legend use the notebook's EPSG:3857 assumption. This is not a geographic basemap.

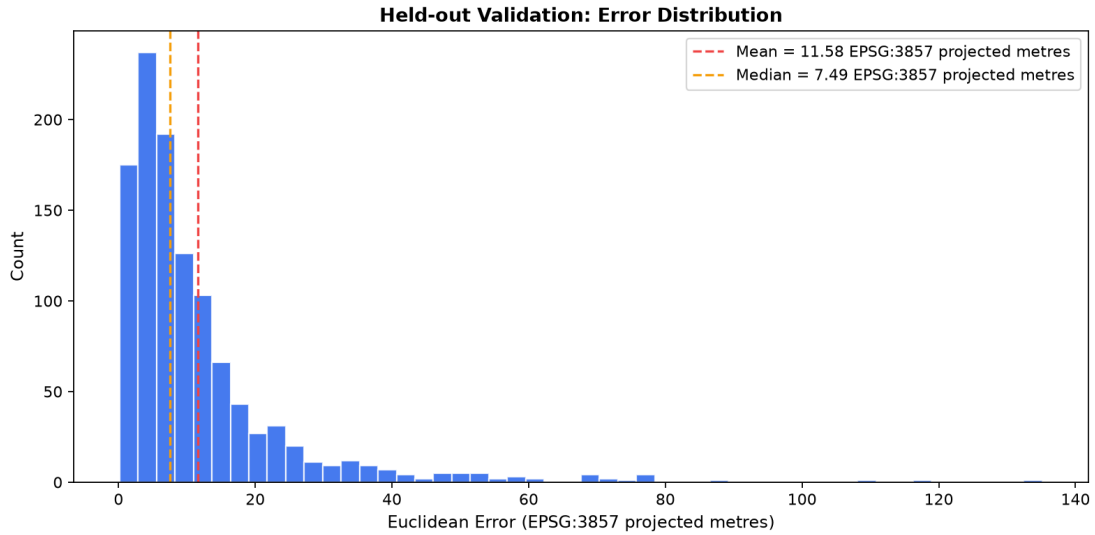


Figure 12: Held-out Euclidean coordinate-error distribution with mean and median marked. A right tail raises the mean above the median.

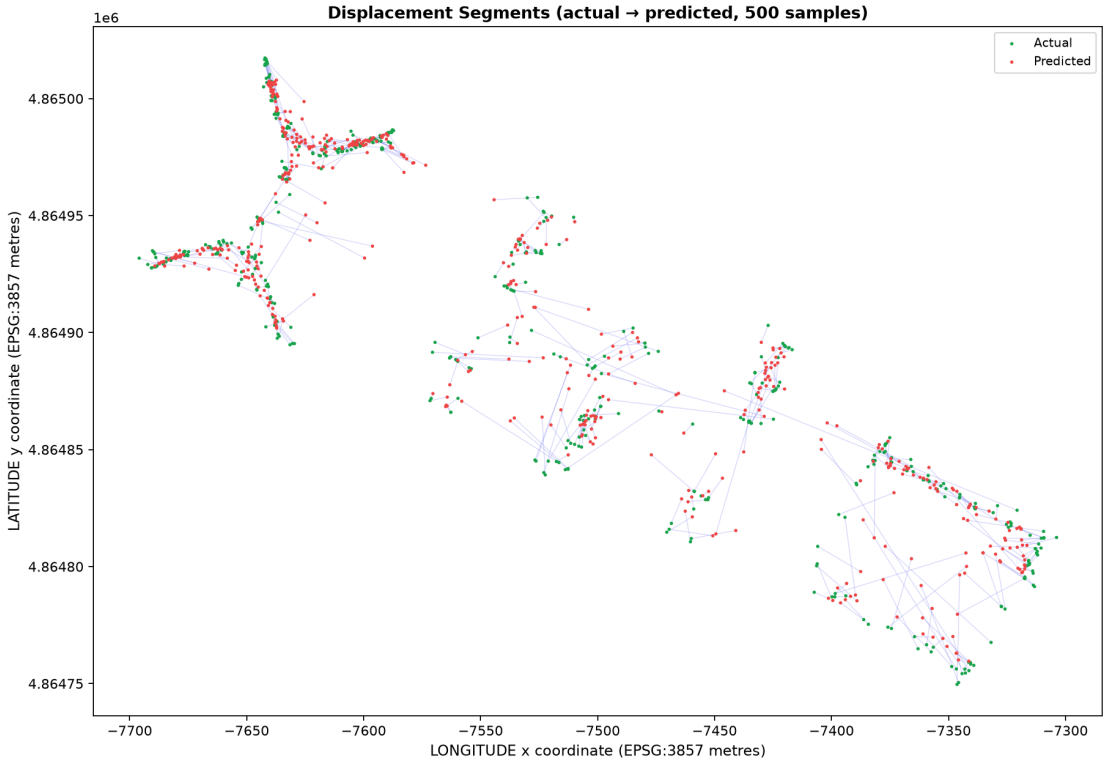


Figure 13: Actual-to-predicted displacement segments for the 500-row sample shown by the coordinate regression notebook. Line length and direction visualize the signed 2D residual; this sample plot complements, but does not replace, metrics over all held-out rows.

8.4 Hierarchical evaluation

Building and floor are predicted independently, so a correct floor code does not by itself imply the correct building–floor location. The integration notebook joins predictions using a composite source-partition and row-ID key, verifies key uniqueness, row coverage and true-label agreement, and recomputes the task metrics from the row-level outputs. It then evaluates exact joint

building–floor correctness. On the 1,111 held-out rows, both labels are correct for 986 records (88.75%).

Among held-out rows with both labels correct, mean coordinate error is 10.00 coordinate units. For 122 rows with a correct building but incorrect floor, it is 23.27 units; the two rows with an incorrect building but correct floor have mean error 44.31; one row with both labels incorrect has error 77.04. The latter groups are very small, so their means are descriptive only. Figure 14 summarizes the hierarchical outcome proportions.

Table 7: Held-out hierarchical outcomes and coordinate error by classification correctness.

Building/floor outcome	Rows	Share	Mean coordinate error
Both correct	986	88.75%	10.00
Building correct, floor wrong	122	10.98%	23.27
Building wrong, floor correct	2	0.18%	44.31
Both wrong	1	0.09%	77.04

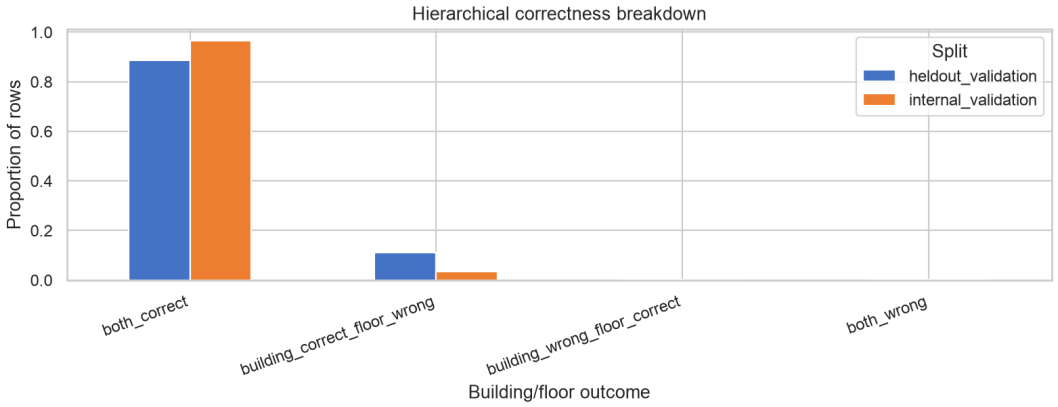


Figure 14: Proportion of rows in each joint building/floor correctness category for internal and held-out validation.

9 Notebook-by-notebook implementation

This section describes the concrete handoff between the five AML notebooks. Each model notebook can run from a fresh kernel and reconstructs the feature allowlist from the canonical CSV schema. Notebook outputs are saved as CSV or PNG artifacts so the integration notebook can validate and consume them without relying on in-memory notebook state.

9.1 ML_01: data preparation, EDA, and feature export

The first notebook is the source of truth for raw input validation and the shared feature contract. It locates the two raw CSV files relative to the repository, checks that their actual columns contain the expected WAP, target, and metadata fields, discovers WAP columns by their numeric suffix, and records row counts, missingness, duplicates, signal ranges, label distributions, and all-absent rows. It treats sentinel 100 as absence. Exact duplicate rows and all-absent training rows are retained because the implementation does not assume they should be removed.

It then creates the single grouped 80/20 split from the training/reference rows. The split uses seed 42 and the building/floor/space composite key, so the downstream tasks use precisely the

same train and internal-validation row sets. The supplied validation file is loaded for descriptive EDA and final feature export, but those records do not contribute to the scaler fit or the development split. This distinction is important: descriptive plots of a held-out file do not constitute model selection, provided no choice is changed in response to them.

The notebook derives each per-WAP mean and population standard deviation from observed internal-training values only. A WAP with zero or undefined variance uses scale 1; a WAP unseen in internal training uses the internal-training pooled signal statistics. Missing standardized values are set below the minimum observed standardized training value, and binary masks retain the true observation state. The notebook builds the six summaries from raw observed RSSI, then concatenates scaled WAPs, masks, and summaries in a fixed order. It stores identifiers, metadata, targets, and source-partition labels alongside the predictors, but places only the 1,046 feature fields in the reusable predictor allowlist.

Three canonical tables are written under `data/processed/`:

- `aml_internal_train_features.csv` (16,085 rows);
- `aml_internal_validation_features.csv` (3,852 rows); and
- `aml_heldout_validation_features.csv` (1,111 rows).

The notebook reloads these CSVs and asserts expected row counts and column order, numeric finite predictors, binary detection masks, target and metadata availability, and the correct source partition. Figure 5 depicts the construction path; Figures 1 and 2 summarize the raw signal and target distributions.

9.2 ML_02: building classification

The building notebook reads the internal-training and internal-validation feature tables first. It reconstructs the predictor list as 520 scaled RSSI columns, 520 presence columns, and six summaries; it separately identifies the building target and the identifiers/metadata used only for alignment. Assertions check compatible schemas, disjoint row IDs, and disjoint building/floor/space groups. The held-out feature path is not loaded for model work until the configuration has been selected and frozen.

A baseline Random Forest is trained on internal training. The candidate loop fits each of the 18 configurations on the same internal-training matrix and scores it on the same internal-validation matrix. The tuning table records each setting and its accuracy and macro F1. The selected setting is refit on internal training only; then the notebook loads the held-out table, verifies the same feature contract, produces one row-level prediction per held-out row, and calculates final accuracy and macro F1. It exports internal-validation and held-out prediction CSVs, a metrics CSV containing baseline/tuned/held-out rows, and a held-out confusion-matrix CSV and figure under `data/tables/` and `data/figures/`.

9.3 ML_03: floor classification

The floor notebook follows the shared feature and split contract, but establishes the floor-class map exclusively from internal-training labels. It sorts the class values and keeps that mapping fixed in every later prediction and confusion matrix. `BUILDINGID`, coordinate targets, and all metadata are excluded from the predictor matrix. Thus, floor classification is intentionally a floor-code prediction task whose errors are considered alongside building predictions later.

The notebook scores an untuned baseline, evaluates the eight fixed parameter combinations on internal validation, and selects by accuracy with macro F1 and deterministic parameter ordering

used for ties. Balanced class weights are enabled for each candidate and the selected estimator. The frozen model is fit on internal training, then evaluated on the held-out data. It writes row-level prediction tables for internal validation and held-out validation, a metrics CSV, a confusion-matrix count table, and a normalized/count visualization. The supported class list is fixed to the training classes so even a rare class remains represented in the output matrix.

9.4 ML_04: coordinate regression

The regression notebook independently checks the same canonical feature columns and prepares a two-column target matrix from longitude and latitude. It creates a strict feature allowlist that excludes coordinate labels and metadata. A baseline multi-output Random Forest is evaluated first. The ten-candidate search then fits on internal training and ranks candidates by mean Euclidean error on internal validation, not by the eventual held-out score. The selected configuration is frozen and refit on internal training before the supplied held-out table is loaded.

The evaluation code computes longitude MAE and latitude MAE separately, plus row-wise Euclidean distances and their mean, median, 75th, and 95th percentiles. It verifies that each output row aligns with its source partition and row ID. The notebook writes two prediction tables, a three-row metrics table (baseline internal validation, tuned internal validation, tuned held-out evaluation), and a ten-row tuning table. It also saves actual-versus-predicted plots, a coordinate-space error scatter, a displacement-segment sample, and error-distribution figures. All maps use recorded coordinates directly and have no geographic basemap.

9.5 ML_05: integrated evaluation and consistency checks

The integration notebook does not train another model. It loads the three task prediction files and canonical feature keys, then joins by the pair (`SOURCE_PARTITION`, `ROW_ID`), because row IDs restart in each source file. Before analysis it verifies unique join keys, full row coverage in each split, matching true labels and coordinates, and exact membership in the expected source partition. It recomputes accuracy, macro F1, axis MAE, and Euclidean metrics from the row-level predictions and checks them against each task's metrics file.

It separately reports internal-validation and held-out outcomes. The hierarchical categories are both building and floor correct, building correct/floor wrong, building wrong/floor correct, and both wrong. It also groups by actual building/floor combinations, regenerates the confusion matrices from integrated rows, and writes consolidated predictions, summary metrics, hierarchy tables, and figures. This validation layer checks that task metrics describe predictions on the same records rather than mismatched CSV rows.

9.6 Artifact locations and reproducibility

The canonical feature tables belong in `data/processed/`. Task predictions, parameter searches, metrics, and confusion counts are stored in `data/tables/`. EDA, confusion, coordinate, and integration plots are stored in `data/figures/`. Relative paths let the notebook sequence run from a clean kernel when the raw dataset files are present. The lockfile targets the verified Python 3.14.2 environment; exact reproducibility also depends on the pinned package versions and the dataset files used for the run.

10 Discussion

The results show that building classification is highly accurate on the supplied held-out partition, while floor classification and coordinate regression are more challenging. Floor accuracy is lower than building accuracy and errors cluster around neighboring floor codes. The coordinate error distribution is spatially structured, and the hierarchical analysis shows higher mean coordinate error for rows with an incorrect floor or building label. These are descriptive associations in the current predictions; the pipeline trains three separate estimators and does not impose a joint probabilistic or geometric constraint among their outputs.

The dramatic difference between development validation and held-out floor/coordinate performance underscores the importance of preserving the provided partition for final reporting. It is not appropriate to describe the held-out performance as generalization to entirely unseen fingerprints or coordinates: repeated signal patterns and repeated 2D coordinate values may cross the existing partitions. The project also reports coordinate errors under an EPSG:3857 assumption that has not been verified from authoritative CRS metadata in the repository.

11 Limitations

- **Partition scope:** the split follows the supplied dataset and the internal building/floor/space grouping protocol. Fingerprint overlap and repeated positions may remain across partitions.
- **Coordinate reference:** EPSG:3857 and metre units are notebook assumptions; a source-of-truth CRS declaration is absent. Projection-scale correction is not applied.
- **Model search:** the searches are compact, fixed grids evaluated on one internal split; there is no repeated cross-validation or uncertainty interval.
- **Separate targets:** building, floor, and coordinates are learned independently. Their outputs can disagree, and no joint constraint enforces a valid building–floor combination or coordinate region.
- **Data imbalance and duplicates:** floor representation is uneven, particularly for floor 4. Exact duplicate rows and all-absent training examples were retained; their influence is not separately ablated.
- **Algorithm comparison:** Random Forest is the implemented estimator family. The reported results do not establish that it is better than XGBoost or other alternatives.

12 Conclusion

The implemented pipeline uses train-only RSSI scaling and explicit missing-signal features, followed by three separately tuned Random Forest models for building, floor, and two-dimensional coordinates. Integration verifies row alignment and exact joint building–floor outcomes. On the 1,111 supplied held-out rows, building accuracy is 99.73%, floor accuracy is 88.93%, and mean coordinate error is 11.58 units under the notebook's EPSG:3857 metre assumption. These scores characterize the existing partition and do not demonstrate performance on wholly unseen fingerprints or positions. The coordinate reference also remains unverified in the repository.

References

[1] J. Torres-Sospedra, R. Montoliu, A. Martínez-Usó, J. P. Avariento, T. J. Arnau, M. Benedito-Bordonau, and J. Huerta, "UJIIndoorLoc: A new multi-building and multi-floor database for WLAN fingerprint-based indoor localization problems," in *Proc. 2014 International Conference on Indoor Positioning and Indoor Navigation (IPIN)*, Busan, Republic of Korea, 2014, pp. 261–270, doi: 10.1109/IPIN.2014.7275492.

[2] UCI Machine Learning Repository, "UJIIndoorLoc," dataset 310, doi: 10.24432/C5MS59. Available: `https://archive.ics.uci.edu/dataset/310/ujiindoorloc`.

[3] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.